

UNIDADE 2

USO DE ESTRUTURAS DE CONTROL

v1.0

PROGRAMACIÓN

CICLO SUPERIOR DE DAM

Manuel Pacior Pérez



Índice

1 INTRODUCCIÓN.....	3
2 A PROGRAMACIÓN ESTRUTURADA.....	3
3 ESTRUTURAS DE SELECCIÓN CONDICIONAL.....	5
3.1 A sentenza if (selección simple).....	6
3.2 Estrutura if - else.....	6
3.3 Estrutura if - else if – else.....	7
3.4 Estrutura switch.....	8
4 ESTRUTURAS DE BUCLE OU ITERATIVAS.....	10
4.1 Estrutura for.....	10
4.2 Estrutura while.....	11
4.3 Estrutura do – while.....	11
4.4 Estrutura for - each.....	12
5 TIPO DE DATO ARRAY EN JAVA.....	12
5.1 Arrays unidimensionais.....	13
5.2 Arrays bidimensionais.....	14
5.3 Arrays multidimensionais.....	16
6 SENTENZAS DE CONTROL: BREAK E CONTINUE.....	17
6.1 Senteza break.....	17
6.2 Senteza continue.....	18
6.3 Exemplo práctico de break e continue combinados.....	19
7 ÁMBITO DE VARIABLES EN ESTRUTURAS DE CONTROL.....	20
7.1 Ámbito de variables en condicionais.....	20
7.1.1 Ámbito no if.....	20
7.1.2 Ámbito no switch.....	21
7.2 Ámbito de variables en bucles.....	22
7.2.1 Ámbito en bucle for.....	22
7.2.2 Ámbito en bucle while.....	23
7.2.3 Ámbito en bucles aniñados.....	23



1 INTRODUCCIÓN

Para crear unha solución computacional a un problema, necesitamos:

- Comprender con detalle o problema a resolver e o seu contexto.
- Comprender os bloques de construción dispoñibles para expresar a solución.

Calquera problema computacional pódese resolver executando unha serie de accións nunha orde específica.

Denominamos **algoritmo** ao procedemento para resolver un problema en termos de:

- As accións a executar.
- A orde na que se executan esas accións.

Denominamos **control do programa** á orde en que se executan as instrucións dun programa. Un programa informático executase, en grande medida, seguindo unha orde secuencial (na mesma orde na que se escribe), se ben desde a aparición das primeiras linguaxes de máquina se incorporaron instrucións á linguaxe que permitan "romper" esta orde de execución e "saltar" a diferentes seccións do código. Isto é coñecido como transferencia de control.

Durante a década dos 60 observouse como o uso indiscriminado das transferencias de control (o "infame" goto), xeraba notables dificultades tanto na creación como no mantemento de programas, sendo unha limitación para a creación de aplicacións máis complexas.

O traballo de Böhm e Jacopini conseguiu demostrar que esas instrucións non eran necesarias (nin recomendadas) e sentou as bases dun novo paradigma, coñecido como programación estruturada.

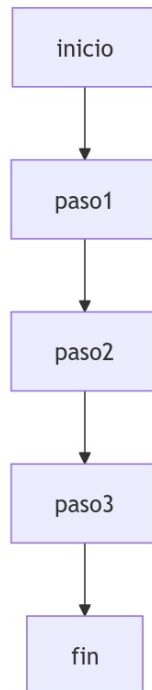
2 A PROGRAMACIÓN ESTRUTURADA

O teorema da programación estruturada de Böhm e Jacopini, formulado en 1966, establece que toda función computable, pode ser implementada nunha linguaxe de programación que combina só tres estruturas lóxicas de control.

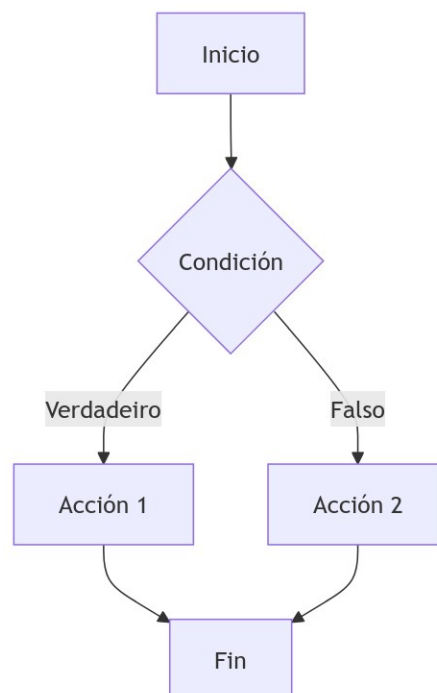


As tres estruturas básicas ás que se refire o teorema da programación estruturada son:

- **Secuencia:** Execución dunha instrución tras outra, segundo a orde de secuencia

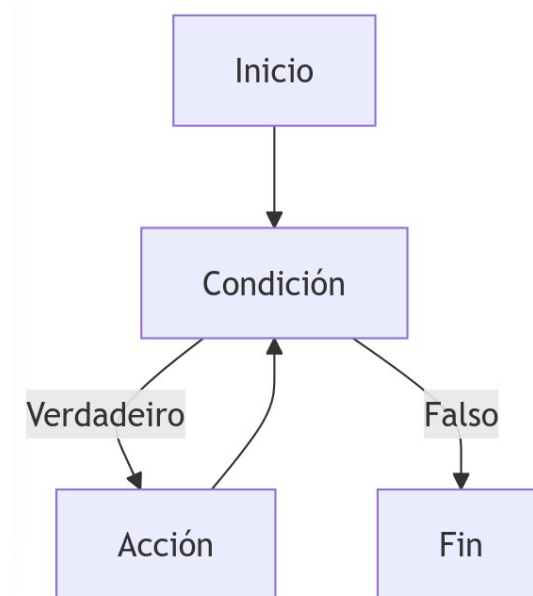


- **Selección (ou decisión condicional):** A estrutura de selección permite tomar unha decisión en función dunha condición, seguindo un camiño distinto en función de se a condición é verdadeira ou falsa.





- **Iteración (ou bucles):** A estrutura de iteración implica a repetición dun conxunto de accións mentres se cumpre unha condición.



Isto significa que non é necesario o uso de instrucións obsoletas como goto ou saltos incondicionais para implementar calquera algoritmo, o que producía código inmanexable, coñecido tamén como “código spaghetti”.

En resumo, o teorema di que calquera programa que pode ser descrito cun diagrama de fluxo pode ser escrito usando só estas tres estruturas fundamentais, o que sentou as bases para a programación estruturada, axudando a mellorar a claridade, mantemento e corrección das aplicacións.

Este teorema tivo un grande impacto na maneira en que se ensina e se practica a programación, xa que fomentou un estilo de programación máis organizado e lexible que foi a base doutros paradigmas posteriores, como a POO (Programación Orientada a Obxectos).

3 ESTRUCTURAS DE SELECCIÓN CONDICIONAL

En Java, as estruturas de selección permiten que o programa tome decisións e execute código condicionalmente en función de se unha condición se cumpre ou non. Os principais tipos de estruturas de selección en Java son:

- **if**
- **if - else**



- **if - else if - else**
- **switch**

A continuación, descríbense en detalle cada unha delas xunto con exemplo de uso para cada caso.

3.1 A sentenza if (selección simple)

A estrutura if avalía unha condición booleana. Se a condición é verdadeira, entón o bloque de código que hai dentro da estrutura if execútase; se é falsa, entón o bloque de código sáltase e non se executa. A súa sintaxe é a seguinte:

```
if (condición) {  
    // Código a executar se a condición é verdadeira  
}
```

Exemplo de uso:

```
public class IfExample {  
    public static void main(String[] args) {  
        int idade = 20;  
  
        if (idade >= 18) {  
            System.out.println("É maior de idade.");  
        }  
    }  
}
```

Neste exemplo, a mensaxe "É maior de idade." imprimírase só se a variable idade é maior ou igual a 18.

3.2 Estrutura if - else

Esta estrutura engade unha acción alternativa no caso de que a condición sexa falsa, é dicir, se a condición do if é verdadeira, o primeiro bloque execútase, se é falsa, execútase o segundo bloque do else.

A súa sintaxe é a seguinte:

```
if (condición) {  
    // Código a executar se a condición é verdadeira  
} else {  
    // Código a executar se a condición é falsa  
}
```



Exemplo de uso:

```
public class IfElseExample {  
    public static void main(String[] args) {  
        int idade = 16;  
  
        if (idade >= 18) {  
            System.out.println("Es maior de idade.");  
        } else {  
            System.out.println("Es menor de idade.");  
        }  
    }  
}
```

Neste caso, como idade é 16, a mensaxe "É menor de idade." será a que se mostre, que é a que está dentro do bloque else.

3.3 Estrutura if - else if - else

Esta estrutura permite comprobar múltiples condicións, se a primeira condición é falsa, avalíanse as seguintes até que unha sexa certa, momento no que se executa ese bloque de código. No caso de que ningunha das condicións sexa certa, entón execútase o bloque else (se está definido, pois non é obrigatorio).

A súa sintaxe é a seguinte:

```
if (condición1) {  
    // Código a executar se a condición1 é verdadeira  
} else if (condición2) {  
    // Código a executar se a condición2 é verdadeira  
} else {  
    // Código a executar se ningunha condición é verdadeira  
}
```

Exemplo de uso:

```
public class IfElseIfExample {  
    public static void main(String[] args) {  
        int nota = 85;  
  
        if (nota >= 90) {  
            System.out.println("Sobresaínte.");  
        } else if (nota >= 70) {  
            System.out.println("Notable.");  
        } else if (nota >= 50) {  
            System.out.println("Aprobado.");  
        } else {  
            System.out.println("Suspenso.");  
        }  
    }  
}
```



```
}  
}  
}
```

No anterior exemplo a variable `nota` vale 85, polo que o programa imprimirá "Notable.", xa que a segunda condición é a verdadeira. Ten en conta que se cambio a orde dos `if`, isto afectará ao seu desempeño, por exemplo:

```
public class IfElseIfExample {  
    public static void main(String[] args) {  
        int nota = 85;  
  
        if (nota >= 50) {  
            System.out.println("Aprobado .");  
        } else if (nota >= 70) {  
            System.out.println("Notable.");  
        } else if (nota >= 90) {  
            System.out.println("Sobresaiñte .");  
        } else {  
            System.out.println("Suspenso.");  
        }  
    }  
}
```

Neste caso a orde dos bloques de control condicional provocará que imprima "Aprobado" pois 85 é maior que 50, cumpre a condición, así que entra no primeiro bloque que cumpre e executa esas instrucións. Ao cumprir esa condición e entrar, xa non comproba as seguintes, polo que é importante ter isto en conta cando definamos a lóxica das nosas estruturas condicionais múltiples.

3.4 Estrutura switch

A estrutura **switch** utilízase cando se queren comparar os valores dunha variable contra múltiples casos:

- Cada **case** compara un valor co da variable e, se coincide, execútase o bloque de código correspondente.
- O default execútase se ningunha das opcións coincide.

É unha estrutura bastante utilizada para realizar control de opcións en menús por consola e cando existen un número de valores discretos que pode tomar unha variable, e queremos controlar para que realice unha ou outra acción en función de cada un.



A súa sintaxe é a seguinte:

```
switch (variable) {  
    case valor1:  
        // Código a executar se variable == valor1  
        break;  
    case valor2:  
        // Código a executar se variable == valor2  
        break;  
    // Máis casos...  
    default:  
        // Código a executar se ningún caso coincide  
}
```

Debemos ter en conta que o uso de **break** é importante para evitar que os bloques de código dos casos posteriores se executen unha vez que xa se comprobou que cumpre un dos casos e se executa.

Exemplo de uso:

```
public class SwitchExample {  
    public static void main(String[] args) {  
        int dia = 3;  
  
        switch (dia) {  
            case 1:  
                System.out.println("Luns");  
                break;  
            case 2:  
                System.out.println("Martes");  
                break;  
            case 3:  
                System.out.println("Mércores");  
                break;  
            case 4:  
                System.out.println("Xoves");  
                break;  
            case 5:  
                System.out.println("Venres");  
                break;  
            default:  
                System.out.println("Fin de semana");  
        }  
    }  
}
```

No exemplo, como a variable dia vale 3, a saída será "Mércores".



4 ESTRUCTURAS DE BUCLE OU ITERATIVAS

En Java, as estruturas iterativas ou de bucle permiten executar repetidamente un bloque de código mentres se cumpre unha condición determinada. As principais estruturas iterativas en Java son:

- **for:** úsase cando coñeces o número exacto de iteracións.
- **while:** execútase mentres a condición sexa verdadeira, e avalíase antes de cada iteración.
- **do-while:** similar ao while, pero avalía a condición despois de cada iteración, garantindo que se execute polo menos unha vez.
- **for-each:** percorre elementos dunha colección (verémolo na segunda avaliación) ou dun array de forma sinxela, sen necesitar xestionar índices.

4.1 Estrutura for

O bucle for úsase cando coñeces o número exacto de iteracións que queres executar. É ideal para recorrer secuencias ou realizar repeticións dun número predefinido de veces.

A súa sintaxe é a seguinte:

```
for (inicialización; condición; actualización) {  
    // Código a executar en cada iteración  
}
```

Exemplo de uso:

```
public class ForExample {  
    public static void main(String[] args) {  
        for (int i = 0; i < 5; i++) {  
            System.out.println("Iteración " + i);  
        }  
    }  
}
```

Neste exemplo, o bucle execútase 5 veces, imprimindo os valores de i desde 0 ata 4.

- **Inicialización:** establece a variable de control (int i = 0).
- **Condición:** indica cando o bucle debe continuar (i < 5).
- **Actualización:** incrementa ou modifica a variable de control ao final de cada iteración (i++).



4.2 Estrutura while

A estrutura while execútase mentres unha condición sexa verdadeira. Úsase cando non se sabe exactamente cantas veces se debe repetir o bucle, pero si se sabe baixo que condición debe continuar.

A súa sintaxe é a seguinte:

```
while (condición) {  
    // Código a executar mentres a condición sexa verdadeira  
}
```

Exemplo de uso:

```
public class WhileExample {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            System.out.println("Iteración " + i);  
            i++;  
        }  
    }  
}
```

Neste caso, o bucle while execútase sempre que i sexa menor que 5, o que fai que o programa imprima os valores de i desde 0 ata 4.

4.3 Estrutura do - while

A estrutura do-while é similar a while, pero a diferenza principal é que o bloque de código sempre se executa polo menos unha vez, xa que a condición é avaliada ao final de cada iteración.

A súa sintaxe é a seguinte:

```
do {  
    // Código a executar en cada iteración  
} while (condición);
```

Exemplo de uso:

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 0;  
        do {  
            System.out.println("Iteración " + i);  
            i++;  
        } while (i < 5);  
    }  
}
```



```
}  
}
```

Aquí, o bucle imprime os valores de *i* desde 0 ata 4. Mesmo se a condición fose inicialmente falsa, o bloque de código executaríase unha vez antes de avaliar a condición.

4.4 Estrutura for - each

A estrutura for-each úsase para recorrer elementos dunha colección (como arrays ou listas) sen a necesidade de xestionar índices de forma explícita. É especialmente útil cando só queres iterar a través dunha colección (verémolas na 2ª avaliación) e arrays (que se explican en detalle no seguinte punto) sen preocuparte pola lóxica de bucle, de xeito que percorre so os elementos que contén.

A súa sintaxe é a seguinte:

```
for (tipo variable : colección) {  
    // Código a executar para cada elemento da colección  
}
```

Exemplo de uso:

```
public class ForEachExample {  
    public static void main(String[] args) {  
        int[] numeros = {1, 2, 3, 4, 5};  
        for (int num : numeros) {  
            System.out.println("Número: " + num);  
        }  
    }  
}
```

Neste exemplo, o bucle for-each percorre todos os elementos do array *numeros* e imprime cada un deles. Non necesitas preocuparte polos índices.

5 TIPO DE DATO ARRAY EN JAVA

En Java, un array é unha estrutura de datos de tamaño fixo que permite almacenar múltiples valores do mesmo tipo nunha soa variable.

As principais características dun array son:

- **Tamaño fixo:** O tamaño dun array establécese no momento da súa creación e non pode cambiar.
- **Indexación:** Os elementos dun array están indexados, é dicir, cada valor dentro



del ten un índice asociado que comeza en 0 e vai até N (sendo N = tamaño do array - 1).

- **Elementos do mesmo tipo:** Todos os elementos dun array teñen que ser do mesmo tipo, sexa primitivo (como int, double) ou de obxectos (como String).
- **Acceso eficiente:** Os arrays permiten acceder aos elementos de forma directa usando o seu índice, pois como cada elemento ten un tamaño fixo, e partimos dunha posición inicial de memoria (que garda o inicio do primeiro elemento), é doado calcular en que posición de memoria se atopan o resto.
 - $\text{PosElementoX} = \text{PosElemento1} + (X * \text{TamañoTipoDato})$

5.1 Arrays unidimensionais

O array unidimensional é o tipo máis sinxelo de array, no que os elementos están almacenados de forma lineal, en posicións que van desde o 0 (primeira posición) até N, a última posición e que se corresponde co [tamaño do array - 1].

Declaración e inicialización dun array unidimensional:

```
// Declaración
tipo[] nomeArray = new tipo[tamaño];

// Inicialización con valores
tipo[] nomeArray = {valor1, valor2, valor3, ..., valorN};
```

Exemplo de uso:

```
public class ArrayExample {
    public static void main(String[] args) {
        // Declaración dun array de 5 enteiros
        int[] numeros = new int[5];

        // Inicialización manual
        numeros[0] = 10;
        numeros[1] = 20;
        numeros[2] = 30;
        numeros[3] = 40;
        numeros[4] = 50;

        // Ou directamente coa inicialización
        int[] outroArray = {10, 20, 30, 40, 50};
    }
}
```



```
// Imprimir os elementos do array
for (int i = 0; i < numeros.length; i++) {
    System.out.println("Elemento en índice " + i + ": " + numeros[i]);
}
}
```

Explicación:

- **int[] numeros = new int[5];** crea un array de 5 enteiros, onde cada elemento está inicializado por defecto a 0.
- Podemos representar o array anterior coa seguinte táboa que indica a posición do elemento e o valor:

Posición	0	1	2	3	4
Valor	10	20	30	40	50

- O bucle for úsase para percorrer o array e imprimir os seus valores.
- **numeros.length** devolve o tamaño do array e a última posición do array, como a comeza en 0 será (numeros.length - 1).

5.2 Arrays bidimensionais

Os arrays bidimensionais, tamén chamados matrices, son arrays que teñen filas e columnas. Cada elemento accédese a través de dous índices: un para a fila e outro para a columna.

Declaración e inicialización dun array bidimensional:

```
// Declaración
tipo[][] nomeArray = new tipo[número_filas][número_columnas];

// Inicialización con valores
tipo[][] nomeArray = {
    {valor11, valor12, valor13},
    {valor21, valor22, valor23},
    {valor31, valor32, valor33}
};
```



Exemplo de uso:

```
public class MatrixExample {
    public static void main(String[] args) {
        // Declaración dun array bidimensional 3x3
        int[][] matriz = new int[3][3];

        // Inicialización manual
        matriz[0][0] = 1;
        matriz[0][1] = 2;
        matriz[0][2] = 3;
        matriz[1][0] = 4;
        matriz[1][1] = 5;
        matriz[1][2] = 6;
        matriz[2][0] = 7;
        matriz[2][1] = 8;
        matriz[2][2] = 9;

        // Outro xeito de inicialización
        int[][] outraMatriz = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Imprimir os elementos da matriz
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz[i].length; j++) {
                System.out.println("Elemento en [" + i + "][" + j + "]: " +
                    matriz[i][j]);
            }
        }
    }
}
```

Explicación:

- **int[][] matriz = new int[3][3];** crea unha matriz de 3 filas e 3 columnas.
- Podemos representar o array anterior coa seguinte táboa que indica as posición do índice horizontal (filas) e vertical (columnas), cun valor de tipo int por cada par:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9



- Hai que usar dous bucles, un para percorrer cada unha das dimensións do array. Neste caso o bucle for anidado úsase para percorrer cada columna, pivotando sobre cada fila, e imprimir os valores asignados a cada par.
- `matriz.length` dá o número de filas, e `matriz[i].length` dá o número de columnas para cada fila `i`.

5.3 Arrays multidimensionais

Java tamén permite crear arrays de máis de dúas dimensións aínda que o seu uso, a nivel práctico, está restrinxido a representar a áreas moi concretas. Por exemplo, un array de tres dimensións pode considerarse como unha táboa de varias matrices.

Declaración dun array tridimensional:

```
// Declaración
tipo[][][] nomeArray = new tipo[dimensión1][dimensión2][dimensión3];
```

Exemplo de uso:

```
public class MultiArrayExample {
    public static void main(String[] args) {
        // Declaración dun array tridimensional 2x2x2
        int[][][] cubo = new int[2][2][2];

        // Inicialización manual
        cubo[0][0][0] = 1;
        cubo[0][0][1] = 2;
        cubo[0][1][0] = 3;
        cubo[0][1][1] = 4;
        cubo[1][0][0] = 5;
        cubo[1][0][1] = 6;
        cubo[1][1][0] = 7;
        cubo[1][1][1] = 8;

        // Imprimir os elementos do array tridimensional
        for (int i = 0; i < cubo.length; i++) {
            for (int j = 0; j < cubo[i].length; j++) {
                for (int k = 0; k < cubo[i][j].length; k++) {
                    System.out.println("Elemento en [" + i + "][" + j + "]"
                        + [" + k + "]: " + cubo[i][j][k]);
                }
            }
        }
    }
}
```




Explicación:

- `int[][][] cubo = new int[2][2][2];` crea un array tridimensional.
- Os bucles for aniñados úsanse para percorrer cada dimensión do array e imprimir os valores correspondentes.

6 SENTENZAS DE CONTROL: BREAK E CONTINUE

En Java, as sentenzas de control `break` e `continue` úsanse para alterar o fluxo de execución dun bucle. Estas sentenzas son especialmente útiles cando traballamos con arrays, xa que permiten manipular a execución dun bucle segundo certas condicións, como a de deter o bucle ou saltar iteracións específicas. Podemos dicir que:

- **break:** Rompe o bucle e salta fóra del completamente.
- **continue:** Salta á seguinte iteración do bucle sen executar o resto do código da iteración actual.

6.1 Sentenza break

A sentenza `break` utilízase para romper ou saír dun bucle antes de que remate normalmente. Cando se atopa un `break`, a execución do bucle detense de inmediato e o control pasa á seguinte liña de código despois do bucle.

Exemplo de uso:

```
public class BreakExample {
    public static void main(String[] args) {
        // Array de enteiros
        int[] numeros = {5, 12, 3, 18, 6, 9};

        // Queremos deter o bucle cando atopemos o número 18
        int buscarNumero = 18;

        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] == buscarNumero) {
                System.out.println("Número " + buscarNumero + " atopado en
índice " + i);
                break; // Rompe o bucle cando o número é atopado
            }
        }

        System.out.println("Bucle rematado.");
    }
}
```



Explicación:

- Neste exemplo, percorremos o array `numeros` buscando o valor 18.
- Cando se atopa o valor 18, imprimimos unha mensaxe e utilizamos `break` para deter o bucle. Non ten sentido continuar a buscar cando xa o atopamos, así que isto fai máis eficiente o noso código.
- Despois de que o `break` se execute, o control pasa á seguinte liña de código fóra do bucle, e o bucle xa non continúa executándose.

6.2 Sentenza continue

A sentenza `continue` úsase para saltar a iteración actual dun bucle e continuar coa seguinte. Cando se atopa un `continue`, o código que segue dentro do bucle para esa iteración é ignorado, pero o bucle segue executándose desde a seguinte iteración.

Exemplo de uso:

```
public class ContinueExample {
    public static void main(String[] args) {
        // Array de enteiros
        int[] numeros = {5, 12, 3, 18, 6, 9};

        // Imprimir só os números impares
        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] % 2 == 0) {
                continue; // Salta os números pares
            }
            System.out.println("Número impar: " + numeros[i]);
        }

        System.out.println("Bucle rematado.");
    }
}
```

Explicación:

- Neste exemplo, percorremos o array `numeros` e utilizamos a sentenza `continue` para saltar os números pares.
- Se un número no array é par (`numeros[i] % 2 == 0`), o `continue` fai que se salte o código restante no bucle e pase directamente á seguinte iteración.
- Só os números impares son impresos.



6.3 Exemplo práctico de break e continue combinados

Podemos combinar ambas sentenzas nun mesmo programa. Por exemplo, imos buscar un número concreto nun array, pero imos saltar os números pares e deter o bucle cando atopemos un número impar específico.

Exemplo de uso:

```
public class BreakContinueExample {
    public static void main(String[] args) {
        // Array de enteiros
        int[] numeros = {5, 12, 3, 18, 6, 9};

        // Queremos saltar os números pares e deter o bucle ao atopar o
        // número impar 9
        int buscarNumero = 9;

        for (int i = 0; i < numeros.length; i++) {
            // Saltar os números pares
            if (numeros[i] % 2 == 0) {
                continue;
            }

            System.out.println("Número impar: " + numeros[i]);

            // Deter o bucle ao atopar o número impar buscado
            if (numeros[i] == buscarNumero) {
                System.out.println("Número " + buscarNumero + " atopado.
Deter o bucle.");
                break;
            }
        }

        System.out.println("Bucle rematado.");
    }
}
```

Saída:

```
Número impar: 5
Número impar: 3
Número impar: 9
Número 9 atopado. Deter o bucle.
Bucle rematado.
```

Explicación:

- Primeiro, utilizamos continue para saltar todos os números pares.
- Despois, cando atopamos o número impar 9, utilizamos break para deter o bucle.



- Deste xeito, só se imprimen números impares e o bucle finaliza unha vez se atopa o número buscado.

7 ÁMBITO DE VARIABLES EN ESTRUCTURAS DE CONTROL

En Java, o ámbito (ou alcance) dunha variable refírese á parte do código na que esa variable é accesible. Este depende de onde foi declarada dentro do programa, e trátase dun concepto importante cando se traballa con estruturas de control.

En base a isto podemos identificar varios tipos de variables, en función do seu ámbito:

- **Variables locais:** Declaradas dentro bloque, só son accesibles nesa parte do código.
- **Variables de instancia (ou de obxecto):** Declaradas dentro dunha clase pero fóra dos métodos, e cada instancia (obxecto) da clase ten a súa propia copia destas variables.
- **Variables estáticas (ou de clase):** Declaradas dentro dunha clase usando a palabra clave static, e compartidas entre todas as instancias da clase.

Nesta unidade imos centrarnos no ámbito de variables locais dentro das estruturas de control de tipo condicional e bucles e podemos definir as seguintes consideracións xerais:

- Unha variable declarada dentro dun bloque, sexa for, while, if, ou switch, só está dispoñible dentro dese bloque.
- As variables declaradas fóra dun bucle ou estrutura de control poden usarse tanto dentro como fóra da estrutura.
- É recomendable declarar variables dentro do ámbito máis pequeno posible, é dicir, o bloque onde se necesiten, para evitar confusións e reducir erros.

7.1 Ámbito de variables en condicionais

Dentro dunha estrutura condicional (if, else, switch): As variables declaradas dentro dun bloque condicional só existen dentro dese bloque. Non se pode acceder a elas fóra del.

7.1.1 Ámbito no if

As variables declaradas dentro dun bloque if ou else só están dispoñibles dentro dese bloque específico e non se poden utilizar fóra del.



Exemplo de uso:

```
public class AmbitoIf {  
    public static void main(String[] args) {  
        int numero = 10;  
  
        if (numero > 5) {  
            String mensaxe = "O número é maior que 5";  
            System.out.println(mensaxe);  
        }  
  
        // Aquí, a variable 'mensaxe' non é accesible porque só existe  
        // dentro do bloque if  
        // System.out.println(mensaxe); // Producirá un erro de compilación  
    }  
}
```

Explicación:

- A variable mensaxe só está dispoñible dentro do bloque if.
- Se intentamos acceder a ela fóra dese bloque, obterás un erro de compilación.

7.1.2 Ámbito no switch

O ámbito dunha variable declarada dentro dun bloque switch está limitado ao caso onde foi declarada. Se se intenta acceder á variable fóra dese caso, obterase un erro.

Exemplo de uso:

```
public class AmbitoSwitch {  
    public static void main(String[] args) {  
        int dia = 2;  
  
        switch (dia) {  
            case 1:  
                String mensaxe1 = "Luns";  
                System.out.println(mensaxe1);  
                break;  
            case 2:  
                String mensaxe2 = "Martes";  
                System.out.println(mensaxe2);  
                break;  
            default:  
                System.out.println("Día descoñecido");  
        }  
  
        // Aquí, 'mensaxe1' e 'mensaxe2' non son accesibles  
        // System.out.println(mensaxe1); // Producirá un erro de  
        // compilación
```



```
// System.out.println(mensaxe2); // Producirá un erro de
compilación
}
}
```

Explicación:

- A variable mensaxe1 ten ámbito dentro do case 1, e a variable mensaxe2 ten ámbito dentro do case 2.
- Ningunha das dúas variables pode ser usada fóra dos seus respectivos casos dentro do bloque switch.

7.2 Ámbito de variables en bucles

7.2.1 Ámbito en bucle for

Dentro dun bucle (for, while, do-while): As variables declaradas dentro dun bucle teñen ámbito limitado ao bloque do bucle. Non se pode acceder a elas fóra do bucle.

No bucle for, a variable de control do bucle (normalmente a variable contador) declárase xeralmente na parte de inicialización do bucle e só está dispoñible dentro do bucle.

Exemplo de uso:

```
public class AmbitoFor {
    public static void main(String[] args) {
        int total = 0;
        // Bucle for con variable local 'i'
        for (int i = 0; i < 5; i++) {
            System.out.println("Dentro do bucle: i = " + i);
            total +=5;
        }
        System.out.println("total = " + total);
        // total é accesible dentro de todo o método main, así que temos
        acceso tamén dentro do bucle e fora del
        // Aquí, a variable 'i' non é accesible e producirá un erro se
        intentamos usala
        // System.out.println("Fóra do bucle: i = " + i); // Producirá un
        erro de compilación
    }
}
```

Explicación:

- A variable total está definida dentro do método principal (main), de xeito que temos acceso a ela desde calquera bloque de código dentro del, tamén dentro do bucle for.



- A variable `i` só está dispoñible dentro do bucle `for`, polo que se tentamos acceder a `i` fóra do bucle, obteremos un erro de compilación. Isto é porque a variable `i` está no ámbito do bucle e non no ámbito global do método.

7.2.2 Ámbito en bucle `while`

Igual que no caso do `for`, as variables declaradas dentro dun bucle `while` só están dispoñibles no ámbito dese bucle.

Exemplo de uso:

```
public class AmbitoWhile {
    public static void main(String[] args) {
        int i = 0; // Variable declarada fóra do bucle

        while (i < 3) {
            int numeroDentroDoBucle = i * 2;
            System.out.println("Dentro do bucle: numero = " +
numeroDentroDoBucle);
            i++;
        }

        // Aquí, a variable 'numeroDentroDoBucle' non é accesible
        // System.out.println(numeroDentroDoBucle); // Producirá un erro de
compilación
    }
}
```

Explicación:

- A variable `numeroDentroDoBucle` ten ámbito dentro do bucle `while`, e non pode accederse a ela fóra do bucle.
- A variable `i`, que foi declarada fóra do bucle, ten ámbito global dentro do método `main`, e pode usarse tanto dentro como fóra do bucle.

7.2.3 Ámbito en bucles anidados

As variables declaradas dentro dun bucle anidado só están dispoñibles dentro dese bucle interno. As variables do bucle externo seguen dispoñibles no bucle interno.

Exemplo de uso:

```
public class AmbitoBucleAnidado {
    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            System.out.println("Bucle externo, i = " + i);
        }
    }
}
```



```
        for (int j = 0; j < 2; j++) {  
            System.out.println("    Bucle interno, j = " + j);  
        }  
    }  
  
    // Aquí, as variables 'i' e 'j' non son accesibles  
    // System.out.println(i); // Producirá un erro de compilación  
    // System.out.println(j); // Producirá un erro de compilación  
}  
}
```

Explicación:

- As variables i e j só son accesibles dentro dos seus respectivos bucles.
- A variable i pertence ao ámbito do bucle externo, e a variable j pertence ao ámbito do bucle interno.
- Ningunha delas pode ser utilizada fóra dos seus bucles. Se descomentamos as dúas últimas liñas veremos que nos dará un erro de compilación.